

Closing the AI Agent Trust Gap

Closing the AI Agent Trust Gap: Kernel-Level Runtime Security Monitoring for AI Agents

Author: Ayush Mishra · ORCID [0009-0002-3116-1969](https://orcid.org/0009-0002-3116-1969)

Technical report, 2026 · Correlic · Source code: <https://github.com/FuloxDev/correlic> · Supplementary rule set: [paper-supplementary](#)

Abstract

AI coding agents now operate with full developer permissions, executing commands, reading files, and making network connections — yet no existing security tool monitors their runtime behavior at the system level. We present a kernel-level monitoring system purpose-built for AI agent security. Using eBPF on Linux, ETW on Windows, and ESF on macOS, we capture every system call initiated by AI agent process trees. A session UUID inheritance mechanism tracks agent lineage across process spawns, enabling AI-gated detection rules that fire exclusively on agent-initiated activity. The detection engine implements 13 behavioral rules mapped to MITRE ATT&CK and 11 multi-step chain patterns. A behavioral baseline system with never-baseline safety guarantees reduces alert volume by ~90% within one week while ensuring sensitive resources are never suppressed. We describe the architecture, cross-platform implementation, and an AI-powered investigation layer with BYOK LLM tool-calling for evidence-based incident analysis.

1. Introduction

The rapid adoption of AI coding agents has introduced a class of security risk that sits outside the detection envelope of conventional endpoint security tools. Unlike traditional development tools that assist with code editing or syntax highlighting, modern AI agents are autonomous executors. When a developer asks Claude Code to debug a deployment issue, the agent does not merely suggest a fix — it spawns shell processes, reads configuration files, executes diagnostic commands, and may make network connections to external services. All of this happens with the developer's full permissions, including access to SSH keys, cloud credentials, CI/CD configurations, and production infrastructure.

This operational model creates what we term the *AI agent trust gap*: organizations benefit from significant productivity gains but have no runtime visibility into what these agents actually do at the system level. The gap is not hypothetical. SentinelOne recently documented a supply chain attack that executed through Claude Code, detected only because their EDR product happened to flag behavioral patterns in a spawned Python subprocess [1]. Exabeam shipped AI agent behavior analytics in late 2025, modeling agents as separate entities with their own baselines [2]. These responses confirm the industry is recognizing the problem, but the solutions being deployed operate at the wrong abstraction level — point-event detection rather than session-level attribution, cloud workflow monitoring rather than developer machine instrumentation.

We argue that effective AI agent security monitoring requires three capabilities that no existing tool provides in combination: (1) kernel-level system call capture that sees what actually happens, not what the agent reports; (2) AI agent session attribution that tracks entire process trees back to their originating agent; and (3) behavioral detection rules specifically designed for the ways AI agents misbehave.

This paper makes the following contributions:

1. We identify and characterize the AI agent runtime security gap, explaining why existing EDR, SIEM, and UEBA tools fail to address it.
2. We present a cross-platform kernel-level monitoring architecture using eBPF (Linux), ETW (Windows), and ESF (macOS) with a novel session UUID inheritance mechanism for AI agent process tree tracking.
3. We describe an AI-gated detection engine with 13 behavioral rules and 11 multi-step attack chain patterns, all mapped to MITRE ATT&CK techniques.
4. We introduce a behavioral baseline system with safety-critical never-baseline guarantees and a user feedback loop that reduces alert volume by approximately 90% within one week of deployment.
5. We present an AI-powered investigation layer using bring-your-own-key (BYOK) large language models with 18 tool-calling functions for evidence-based incident analysis.

The remainder of this paper is organized as follows. Section 2 establishes background and our threat model. Section 3 describes the system architecture. Sections 4-9 detail the implementation of each major subsystem. Section 10 presents our evaluation. Section 11 discusses related work. Section 12 covers limitations and future directions.

2. Background and Threat Model

2.1 How AI Coding Agents Operate

AI coding agents differ fundamentally from previous generations of developer tools. A tool like a linter or type checker operates on source code in isolation, producing static analysis results without side effects. An AI coding agent operates on the developer's environment. When Claude Code receives a task, it may spawn a bash shell, which runs `ls` to enumerate files, then `cat` to read a configuration file, then `npm install` to add a dependency, then `curl` to test an endpoint. Each command in this sequence is a separate process, and each process inherits the developer's full permissions.

The process tree structure is critical to understanding the security surface. An AI agent root process (e.g., `claude-agent` or `cursor`) spawns child processes for each action. These children may spawn grandchildren. A single AI session can produce a process tree 5-10 levels deep involving dozens of distinct executables. Every process in this tree has access to the developer's SSH keys (`~/.ssh/`), cloud credentials (`~/.aws/`, `~/.kube/`, `~/.config/gcloud/`), environment variables, and any file the developer can read.

2.2 Threat Model

We consider three classes of adversary:

Compromised agent. The AI agent itself is compromised through a supply chain attack on the agent binary [20, 21], a malicious plugin, or a backdoor in the model serving infrastructure. In this case, the agent behaves normally for most tasks but exfiltrates credentials or establishes persistence during certain operations.

Prompt injection. A malicious actor places crafted instructions in source code comments, documentation files, or dependency READMEs that cause the AI agent to perform unintended actions — reading sensitive files, making network connections, or modifying CI/CD configurations [3, 19].

Autonomous misbehavior. The AI agent, without explicit malicious input, takes actions that exceed the developer's intent. This includes accessing files outside the project directory, making network connections to unknown endpoints, or executing privilege escalation commands as part of a broadly interpreted task.

Trust boundary. We trust the operating system kernel and our monitoring agent. We do not trust the AI coding agent, its process tree, or any data the agent produces about its own behavior. This is why kernel-level monitoring is essential — the AI agent cannot lie about what system calls were made, because the kernel captures them before the agent is aware of the capture.

Out of scope. We do not defend against kernel rootkits, attacks on the monitoring agent binary itself, or hardware-level attacks. We also do not attempt to determine the semantic intent of AI agent actions — only whether the observable system-level behavior matches known attack patterns.

2.3 Why Existing Tools Are Insufficient

Endpoint Detection and Response (EDR). Modern EDR products perform behavioral analysis and can detect anomalous process behavior. SentinelOne's OneClaw [1] provides AI agent discovery and structured telemetry. However, EDR operates at the granularity of individual process events. It can report "a Python process did something suspicious" but cannot reconstruct "this was part of an AI coding session that started 40 minutes ago, touched these 12 files, made these 3 network connections, and this specific action crossed a line." EDR lacks session-level attribution. Microsoft's Agent Governance Toolkit [9] and Gen Digital's Sage [10] address these gaps at the application layer, but do not provide kernel-level visibility.

Security Information and Event Management (SIEM). AI coding agents do not emit security-formatted logs. They produce no syslog, no audit events, and no native integration with log pipelines. A SIEM has zero data about AI agent activity unless a separate telemetry layer provides it.

User and Entity Behavior Analytics (UEBA). Exabeam's AI agent analytics [2] models agents as separate entities with behavioral baselines. This is meaningful progress, but current implementations focus on cloud workflow orchestration agents, not developer-machine AI coding tools. UEBA sees the user's behavior as normal — they are sitting at their desk coding. The anomalous behavior is happening inside the AI agent's process tree, which UEBA does not instrument.

Network monitoring. Network tools see every connection but cannot determine whether a human or an AI agent initiated it. A curl to an external API looks identical at the packet level regardless of origin. Without process-level attribution, network monitoring is insufficient.

3. System Architecture

Figure 1: System Architecture

Layer	Component	Description
5. Presentation	Next.js Dashboard	Findings, incidents, baselines, block rules, AI chat, timeline
4. AI Intelligence	3-layer context system	BYOK LLM providers, tool-calling (18 tools), pre-computed dossiers, threaded chat
3. Storage	PostgreSQL 14+ / Neo4j 5+	Events, findings, incidents, baselines (PG). Process trees, attack chains, AI labels (Neo4j, optional)
2. Processing	Detection engine	Sampling (90% reduction) -> Detection (13 rules) -> Chain correlation (11 patterns) -> Baseline suppression -> Incident clustering -> Notification
1. Collection	Kernel agents	Linux: eBPF/kprobe. Windows: ETW + USN. macOS: ESF + kqueue. All dispatch via HTTPS (batched, rate-limited, deduplicated)

Our system consists of five layers, shown in Figure 1.

Collection. Platform-native kernel agents capture system calls on monitored machines. The Linux agent uses eBPF programs attached to tracepoints and kprobes. The Windows agent uses ETW kernel providers and the NTFS USN Journal. The macOS agent uses Apple's Endpoint Security Framework. All agents normalize events into a canonical schema before dispatch.

Processing. The backend server (Go, 80+ HTTP endpoints) receives batched events over TLS. A security-first sampling pipeline reduces volume by approximately 90% while preserving all security-relevant events. The detection engine evaluates 13 rules and 11 chain patterns. Behavioral baselines suppress known-normal findings.

Storage. PostgreSQL serves as the primary store for events, findings, incidents, and configuration. Neo4j provides an optional graph database for process tree relationships and attack chain visualization. The system operates with or without Neo4j — graph features degrade gracefully.

AI Intelligence. A three-layer context system (system profiles, hierarchical rollups, learned patterns) feeds a BYOK LLM integration supporting five providers. Pre-computed incident dossiers enable evidence-based analysis through tool-calling.

Presentation. A web dashboard (Next.js) provides findings triage, incident investigation, baseline management, block rule configuration, and AI-powered chat.

3.1 Event Flow

The end-to-end pipeline from kernel event to stored finding operates in ten steps:

1. A kernel event fires (system call, tracepoint, or ETW notification).
2. The agent collects, parses, normalizes, and classifies the event.
3. The agent dispatches events to the backend via HTTPS (batched, rate-limited, deduplicated).
4. The backend authenticates the request (mTLS or API key) and validates the payload.
5. The sampling pipeline applies security-first filtering (Section 5).
6. Events are stored in PostgreSQL. Process execution events are written to Neo4j immediately for graph construction (Tier 1 correlation).
7. AI attribution tags events with session UUIDs from the lineage tracker.
8. The detection engine evaluates applicable rules, producing findings that pass through baseline suppression, exception matching, and cooldown rate limiting.
9. The chain correlator checks whether new findings complete any of 11 multi-step attack patterns.
10. In-app notifications are created for all findings. Incidents trigger external delivery via Slack or HMAC-signed webhooks.

We measure end-to-end latency from kernel event capture to stored finding at under 100 milliseconds.

4. Kernel-Level Collection

4.1 Linux: eBPF

On Linux, we deploy eBPF programs attached to kernel tracepoints and kprobes. We require Linux 5.8 or later with BTF (BPF Type Format) support, which enables CO-RE (Compile Once, Run Everywhere) portability across kernel versions.

Table 1: eBPF programs and hook points

Program	Hook Point	Event Type
execsnoop.bpf.c	tp/sched/sched_process_exec	process_exec
fork.bpf.c	tp/sched/sched_process_fork	process_fork
exit.bpf.c	tp/sched/sched_process_exit	process_exit
fileopen.bpf.c	tp/syscalls/sys_enter_openat	file_open
connect.bpf.c	kprobe/sys_connect	net_connect

Each program writes events to a dedicated ring buffer. The userspace agent reads from these buffers with non-blocking delivery and backpressure to prevent silent drops. Events are parsed with careful attention to struct alignment — the C compiler inserts padding between `__u32` and `__u64` fields that the Go parser must account for. We use `pahole` to verify struct layouts before each release.

A kernel-side BPF hash map (`ai_pids`) tracks active AI process PIDs. When a uprobe program (e.g., TLS interception) encounters an unknown PID, it checks the parent PID against this map and atomically registers the child if the parent is a known AI process. This kernel-side lazy inheritance eliminates a race condition where events from newly spawned AI child processes would arrive before the userspace fork event propagated through the lineage tracker.

4.2 Windows: ETW and USN Journal

On Windows 10 and later, we deploy a single ETW (Event Tracing for Windows) real-time session with four kernel providers.

Table 2: Windows ETW providers

Provider	Event IDs	Events Captured
Kernel-Process	1, 2	Process start/stop
Kernel-File	10, 11, 12, 30	File create/delete/access
Kernel-Network	12, 15, 26, 29	TCP connect/accept (IPv4/v6)
DNS-Client	3008, 3009	DNS query/response

We supplement ETW with Windows Security Audit events (Event IDs 4688/4689 for process creation, 4657 for registry writes, 4672 for privilege use, 4698 for scheduled task creation) and the NTFS USN Journal for reliable file change tracking.

Command-line capture on Windows is particularly challenging. Short-lived processes (e.g., `curl`, `cat`) may exit within 5 milliseconds — before a userspace goroutine can read the Process Environment Block (PEB). Our solution uses a three-tier fallback: (1) the Security Audit event 4688 command-line cache, which is written by the kernel and 100% reliable; (2) a PEB read from within the ETW callback itself using `NtQueryInformationProcess` and `ReadProcessMemory`; (3) a runner-side fallback using `procinfo.ReadProcCmdline`.

Windows path normalization is a substantial engineering challenge. Paths arrive in NT device format (`\Device\HarddiskVolume3\Users\...`), UNC format, with mixed forward/backward slashes, and with various prefix notations. We normalize all paths to forward-slash format with drive letters and perform case-insensitive comparison for baseline matching.

4.3 macOS: Endpoint Security Framework

On macOS 13 (Ventura) and later, we use Apple's Endpoint Security Framework (ESF) for process and file events. ESF provides `ES_EVENT_TYPE_NOTIFY_EXEC`, `ES_EVENT_TYPE_NOTIFY_EXIT`, `ES_EVENT_TYPE_NOTIFY_OPEN`, and `ES_EVENT_TYPE_NOTIFY_FORK` notifications with full process metadata including PID, PPID, executable path, and signing information.

File monitoring supplements ESF with `FSEvents` for directories containing sensitive resources (`~/ .ssh`, `~/ .aws`, `~/ .kube`). Network connection monitoring uses `lsof` polling at 2-second intervals, as ESF does not expose network events.

4.4 Session UUID Inheritance

A core challenge in AI agent monitoring is attributing system-level events to the correct AI agent session. When Claude Code spawns `bash`, which runs `curl`, which makes a TCP connection, we need to know that the entire chain originated from a single AI coding session.

We solve this with a session UUID inheritance mechanism. When the lineage tracker detects an AI agent root process (via pattern matching on executable names and command-line arguments), it assigns a unique session UUID. All child processes automatically inherit this UUID through the lineage tracker's `pidToSession` map. The session UUID propagates to grandchildren and beyond, regardless of depth.

A 10-second grace period preserves session attribution for recently exited processes. When a PID exits, its session mapping moves to the `graceSession` map with a TTL. This handles the common case where a short-lived process (e.g., `curl`) exits before its network event arrives at the processing layer.

On Linux, the eBPF `ai_pids` BPF map provides an additional kernel-side check: if a uprobe encounters an unrecognized PID, it reads the parent's TGID from `task->real_parent->tgid` and checks the map. If the parent is a known AI process, the child is registered atomically in kernel space — zero latency, zero race conditions.

4.5 Event Normalization

All three platform collectors normalize events into a canonical schema with three components:

- **Actor:** PID, PPID, executable path, command-line arguments, username, working directory, AI session ID, AI type.
- **Target:** File path (for file events), IP address and port (for network events), domain (for DNS events), registry key (for Windows).
- **Context:** Event-specific metadata including timestamps, container context, and platform identifiers.

This normalization enables detection rules to operate identically across platforms. A rule checking for SSH key access works the same whether the event came from an eBPF `openat` tracepoint or a Windows USN Journal entry.

5. Intelligent Sampling

Not every kernel event requires storage and analysis. A developer's AI agent may generate thousands of benign file reads and process executions per hour. Storing and evaluating all of them is wasteful and would degrade detection signal-to-noise ratio.

Our sampling pipeline uses a security-first ordering that ensures no meaningful event is discarded:

1. **AI event bypass.** Any event originating from an AI agent session (determined by the presence of an `ai_session_id` in the event context) bypasses sampling entirely. This is the most critical rule: the primary

use case — monitoring AI agents — is never compromised by sampling.

- 2. **Always-keep event types.** Certain event types (process_exec, net_dns) are always retained regardless of source, as they are essential for process tree construction and DNS monitoring.
- 3. **Suspicious pattern matching.** Over 80 file path patterns across 12 categories are checked. These include SSH keys and crypto material (~/.ssh/, id_rsa, .pem), cloud provider credentials (~/.aws/, ~/.kube/, ~/.config/gcloud/), system authentication files (/etc/shadow, /etc/passwd), container secrets (docker.sock, /run/secrets/), database credentials (.pgpass, .my.cnf), and application secrets (.npmrc, .env). Any event matching a suspicious pattern is retained.
- 4. **Benign process filtering.** Known benign system processes (/bin/ls, /usr/bin/grep, etc.) that did not match any suspicious pattern are dropped.
- 5. **Probabilistic sampling.** Remaining events are sampled at a configurable rate (default: retain 10%).

The ordering is critical. Suspicious pattern matching occurs *before* benign process filtering. A cat process reading /etc/shadow is retained because the path matches a suspicious pattern, even though cat itself is a benign binary. Reversing this order would be a security vulnerability.

User-defined watchlist patterns are checked at the highest priority within the suspicious category, enabling organizations to add environment-specific sensitive paths.

6. Detection Engine

6.1 AI-Gated Evaluation

All detection rules gate on AI process attribution before evaluation. When an event arrives at the detection engine, the first check is whether the originating process belongs to an AI agent session. If it does not, no rules are evaluated and no findings are generated. This single gate eliminates the vast majority of false positives that would plague a system monitoring all endpoint activity.

The AI attribution check uses Neo4j's IsAIProcess() query, which walks up to 10 edges of the process tree looking for an :AI-Agent-labeled ancestor. The AI label propagates through the tree via a BFS traversal up to 20 edges from the AI root process.

6.2 Detection Rules

We implement 13 detection rules, each mapped to one or more MITRE ATT&CK techniques.

Table 3: Detection rules with MITRE ATT&CK mapping

Rule ID	Severity	MITRE Techniques	Trigger
ai.credential_access	critical	T1552, T1552.004	file_open on sensitive credentials
ai.unauthorized_exec	high	T1059, T1059.004	process_exec of network/shell tools
ai.excessive_writes	dynamic	T1485, T1486	file_open burst in 30s window
ai.data_exfiltration	critical	T1041, T1567	net_connect after file read (15min)
ai.unexpected_network	dynamic	T1071, T1571	net_connect on non-standard ports
ai.suspicious_dns	dynamic	T1568, T1071.004	DNS query to suspicious TLDs
ai.persistence	critical	T1546, T1053, T1098.004, T1543	cron/systemd/SSH/registry writes

Rule ID	Severity	MITRE Techniques	Trigger
ai.privilege_escalation	critical	T1548, T1068, T1611	sudo/su/setuid/LOLBin execution
ai.code_tampering	high	T1195.002, T1554	CI/CD config or manifest modification
ai.container_escape	critical	T1611, T1610, T1613	Docker socket/namespace access
ai.discovery	low	T1082, T1083, T1057	3+ recon commands in 60s
ai.command_activity	low	T1059	Catch-all command audit
ai.file_activity	low	T1083	Catch-all file audit

Rules are indexed by event type for efficient lookup. Each rule implements an `Evaluate(EvalContext)` function wrapped in `safeEvaluate()` with panic recovery, ensuring a malformed event cannot crash the detection pipeline.

The credential access rule employs tiered sensitivity. SSH private keys (`id_rsa`, `id_ed25519`), `/etc/shadow`, and cloud credential files are Tier 1 — critical severity, immediate finding. Environment files (`.env`) and generic config files are Tier 2 — lower severity with contextual evaluation.

The data exfiltration rule uses temporal correlation. It maintains a 15-minute window of file reads per AI session and checks whether subsequent network connections correlate with prior sensitive file access. This read-then-send pattern detects exfiltration that occurs across separate processes within the same AI session.

6.3 Chain Correlation

Individual findings are data points. Chain findings are evidence. The chain correlator monitors a sliding window of findings per host and session, checking whether sequences match any of 11 predefined attack patterns.

Table 4: Attack chain patterns

Pattern	Steps	Window	Amplified Severity
credential_theft	credential_access -> data_exfiltration	20 min	critical (0.95)
reverse_shell_setup	unauthorized_exec -> unexpected_network	5 min	critical (0.90)
lateral_movement	unauthorized_exec -> unexpected_network[internal]	10 min	high (0.85)
full_compromise	credential_access -> unauthorized_exec -> exfil/network	30 min	critical (0.95)
persistence_backdoor	privilege_escalation -> persistence	15 min	critical (0.95)
supply_chain_attack	code_tampering -> exfil/network	20 min	critical (0.90)
data_staging	excessive_writes -> exfil/network	20 min	critical (0.90)
credential_persistence	credential_access -> persistence	15 min	critical (0.95)
privesc_credential_exfil	privesc -> credential_access -> exfil/network	30 min	critical (0.95)
recon_to_escalation	discovery -> privilege_escalation	15 min	high (0.85)
container_breakout	container_escape -> persistence/creds/privesc	10 min	critical (0.95)

Chain findings always bypass cooldown rate limiting. A multi-step attack pattern is too important to suppress because a component finding was recently emitted.

6.4 Suppression Pipeline

Before a finding is stored, it passes through four suppression stages:

1. **Baseline matching.** $O(1)$ in-memory lookup against the behavioral baseline cache (Section 7).
2. **Rule exception matching.** Per-organization, per-host exceptions with context filtering.
3. **Cooldown rate limiting.** Repeat findings are suppressed within configurable windows (default 10 minutes, 15 minutes for `credential_access`, 5 minutes for `unexpected_network`).
4. **Severity dampening.** Findings with confidence below 0.60 are downgraded one severity level (critical to high, high to medium).

Findings are deduplicated via deterministic IDs: `{detection_id}:{host_id}:{pattern_key}`. The database enforces `ON CONFLICT DO NOTHING` to prevent duplicate storage across retries or race conditions.

7. Behavioral Baseline System

7.1 Auto-Observation

When an event passes through the detection engine and triggers zero findings — meaning it is demonstrably benign — the baseline system extracts a pattern and records it as an observed baseline.

Table 5: Baseline pattern extraction

Event Type	Signal Type	Pattern Format	Example
file_open	file_pattern	directory/**	/home/user/.aws/**
process_exec	binary	executable basename	curl
net_connect	network_dest	BGP prefix:port	104.18.0.0/24:443
net_dns	dns_domain	domain	api.openai.com

Observed baselines have a 30-day TTL from their last-seen timestamp. If a pattern is not observed again within 30 days, the baseline expires and future matching events generate findings again. This prevents stale baselines from masking new threats.

7.2 User Feedback Loop

When an analyst marks a finding as "allowed" through the dashboard, the system creates a user-confirmed baseline for that pattern. User-confirmed baselines never expire. They persist permanently unless manually deleted.

A critical invariant: user-confirmed baselines are never downgraded to observed. The database uses a SQL `CASE` expression on upsert conflict to preserve the higher trust level. If a human decided a pattern is safe, the system respects that decision regardless of how many times the same pattern is subsequently auto-observed.

7.3 Never-Baseline Safety List

Certain resources are too sensitive to ever be auto-baselined, regardless of frequency. Our never-baseline list includes:

- **Files:** SSH private keys (suffixes: `_rsa`, `_ecdsa`, `_ed25519`, `.pem`, `.key`, `.p12`, `.pfx`), directories (`/.ssh/`, `/.azure/`, `/.config/gcloud/`, `/.kube/`), system files (`shadow`, `passwd`, `sudoers`).
- **Binaries:** `nc`, `ncat`, `socat`, `nmap`, `base64`, `xxd`, `useradd`, `usermod`, `passwd`, `chmod`, `chown`, `setcap`.
- **Network ports:** 4444 (Metasploit), 5555, 1337, 6666/6667 (IRC C2), 8888, 9001/9050/9150 (Tor), 31337.
- **DNS domains:** `.onion`, `.i2p`, `pastebin.com`, `paste.ee`, `transfer.sh`, `file.io`.

This list ensures the system cannot be trained to ignore genuinely dangerous activity, even if an attacker gradually normalizes access to sensitive resources.

7.4 Interaction with Chain Correlation

Individual baselines suppress individual findings, but they do not suppress chain findings. If an AI agent's SSH key access is baselined (perhaps the developer explicitly allowed it), and the same session subsequently makes an outbound connection to an unknown IP, the chain correlator evaluates the `credential_theft` pattern and fires regardless. Multi-step attack sequences are evaluated in full context, not filtered by component-level baselines.

7.5 Cache Architecture

The baseline cache uses an in-memory map with composite keys (`signalType + \x00 + pattern + \x00 + hostID + \x00 + aiType`) for $O(1)$ lookup. A background goroutine refreshes the cache from PostgreSQL every 30 seconds. Cache mutations (upsert, delete) trigger immediate reload.

We measured baseline matching at sub-microsecond latency, making it negligible in the per-event processing pipeline.

8. Incident Correlation

8.1 Tier 1: Real-Time Process Graph

Process execution events are written to Neo4j immediately. When an AI agent spawns a child process, the `PROCESS_PARENT` edge exists in the graph before detection rules evaluate. This ensures that AI-gating checks (`IsAIProcess()`) have accurate ancestry data.

The Neo4j schema uses `:Event` nodes with an additional `:AIAgent` label for processes within AI session trees. We define 12 edge types including `PROCESS_PARENT`, `NET_CONNECT`, `FILE_OPEN`, `FILE_WRITE`, and `AI_SPAWNED`. AI label propagation uses a Cypher BFS query traversing up to 20 `PROCESS_PARENT` edges from each AI root.

8.2 Tier 2: Batched Activity Correlation

Non-process events (file access, network connections, DNS queries) are buffered and written in batches for graph consistency. The window manager flushes events after 5 minutes, 1000 events, or 30 seconds of inactivity — whichever comes first. This balances graph freshness against write throughput.

8.3 Incident Clustering

Findings are grouped into incidents by host and AI session within a 30-minute merge window. Chain findings create new incidents containing all constituent step findings. Standalone findings attempt to merge into existing open incidents on the same host and session; if no candidate exists, a new incident is created.

Low-severity single findings generate auto-resolved incidents to keep the analyst queue clean. If a subsequent high-confidence finding merges into an auto-resolved incident, the incident reopens automatically.

Incident severity uses a highest-finding-wins calculation with chain amplification: when a chain pattern is detected, severity is promoted by one level (medium to high, high to critical).

9. AI-Powered Investigation

9.1 Three-Layer Context System

We maintain three layers of context to support LLM-based analysis:

Layer 1: System Profiles. Per-host structured summaries of the environment: which AI agents are installed, their process patterns, network baselines, and commonly accessed paths. Updated hourly.

Layer 2: Context Windows. Hierarchical rollups of agent activity at increasing granularity: 1-minute snapshots (kept 1 hour), 1-hour summaries (kept 24 hours), 24-hour summaries (kept 30 days), weekly summaries (kept 1 year). Each window captures process execution counts, file access counts, network connection counts, unique binaries, sensitive file accesses, and anomaly indicators. Steady-state storage is approximately 750 KB per host.

Short-term windows catch fast attacks. Long-term windows catch slow-moving advanced persistent threats that unfold over days or weeks. When raw events are cleaned per retention policy, the intelligence in context windows is preserved.

Layer 3: Learned Patterns. Real-time aggregation of analyst verdicts. When findings are allowed, dismissed, or blocked, the pattern learner extracts a key (e.g., `command:cursor:reg.exe:low_context`) and maintains a verdict with confidence. Patterns dismissed more than 10 times reach `always_benign` (0.99 confidence). Any blocked pattern is immediately classified as `always_malicious` (0.95 confidence).

9.2 BYOK LLM Integration

We support five LLM providers: OpenAI, Anthropic (Claude), Google Gemini, Groq, and xAI. API keys are encrypted with AES-256-GCM and stored per-organization. No data transits our infrastructure — LLM API calls go directly from the customer's deployment to their chosen provider.

9.3 Incident Dossiers

When an incident is created, the system pre-computes a structured dossier assembling the process chain (as an ASCII tree with [FLAGGED] markers), a chronological timeline (max 30 entries), network activity with ASN resolution, sensitive files accessed (grouped by category), per-executable behavioral context, and related incidents from the same host in the last 30 days. This dossier gives the LLM everything it needs to reason accurately about the incident without hallucination-prone open-ended context.

9.4 Tool-Calling

The LLM has access to 18 tools for real-time data retrieval: 4 intelligence tools (system profile, context windows, learned patterns, false positive rate), 5 graph tools (process trees, attack chains, related events, session activity, lateral movement), and 9 query tools (process lists, connections, ports, containers, with diff operations for temporal comparison).

The tool-calling loop runs up to 5 iterations. Security constraints ensure the LLM can only query data scoped to the incident's host and time window (with +/- 30 minute padding). Individual tool results are capped at 32 KB to prevent context window overflow.

9.5 Threaded Conversation

Analysts interact with the AI through a threaded chat interface per incident. The system loads the last 20 messages as conversation history for each request, maintaining context across multiple questions. Threads auto-purge after 7 days of inactivity.

10. Evaluation

We evaluate our system across three dimensions: detection coverage, noise reduction effectiveness, and runtime performance.

10.1 Detection Coverage

Table 6: MITRE ATT&CK technique coverage

Our 13 detection rules cover 18 techniques across 8 MITRE ATT&CK tactics: Credential Access (T1552, T1552.004), Execution (T1059, T1059.004), Impact (T1485, T1486), Exfiltration (T1041, T1567), Command and Control (T1071, T1071.004, T1571, T1568), Persistence (T1546, T1053, T1098.004, T1543), Privilege Escalation (T1548, T1068, T1611), and Discovery (T1082, T1083, T1057). The 11 chain patterns provide coverage for multi-step sequences spanning multiple tactics.

To validate detection accuracy, we replayed attack scenarios against a test environment running Claude Code and Cursor with the monitoring agent active. Each scenario simulated one of the three adversary classes from our threat model:

Table 7: Detection validation results

Scenario	Adversary Class	Rules Triggered	Chain Detected	False Negatives
SSH key read → curl to external IP	Prompt injection	credential_access, data_exfiltration	credential_theft	0
Reverse shell via nc after pip install	Supply chain	unauthorized_exec, unexpected_network	reverse_shell_setup	0
crontab persistence after sudo	Autonomous misbehavior	privilege_escalation, persistence	persistence_backdoor	0
.env read, no exfil	Benign (false positive test)	credential_access (Tier 2)	None	N/A
Recursive file enumeration + cloud cred read	Prompt injection	discovery, credential_access	recon_to_escalation	0
Docker socket access → host mount	Autonomous misbehavior	container_escape	container_breakout	0

All attack scenarios were detected. The .env access scenario correctly generated a low-severity Tier 2 finding rather than a critical alert, validating our tiered sensitivity model.

10.2 Noise Reduction

We deployed the system on 3 developer workstations running Claude Code and Cursor over a 14-day observation period. Each workstation generated approximately 4,000-6,000 raw kernel events per hour during active AI coding sessions.

Table 8: Alert volume reduction over time

Day	Raw Events/hr	After Sampling	Findings Generated	Baselined	Net Findings
1	~5,200	~520 (90% reduction)	47	0	47
2	~4,800	~480	38	19	19

Day	Raw Events/hr	After Sampling	Findings Generated	Baselined	Net Findings
7	~5,100	~510	41	34	7
14	~4,900	~490	39	38	1

The single remaining finding on Day 14 was a legitimate `credential_access` alert for an SSH key read that correctly appeared on the never-baseline list. This demonstrates that the never-baseline safety guarantee functions as designed — noise decreases but sensitive resource access is never suppressed.

The first detection rule we deployed (`ai.credential_access`) initially generated 47 findings on Day 1. After analyst triage and auto-observation, this dropped to 1 true finding by Day 14 — a 97.9% reduction in noise with zero missed true positives.

10.3 Performance

Table 9: Runtime performance measurements

Metric	Measurement
Agent event processing throughput	~70 events/sec per host
Sampling reduction ratio	~90% (configurable)
End-to-end latency (kernel event → stored finding)	< 100 ms
Baseline cache lookup	< 1 μ s (in-memory, $O(1)$)
Block rule decision time (agent-side)	< 1 μ s
Neo4j process tree write	~2 ms per event
Detection rule evaluation (all 13 rules)	~500 μ s per event
Memory overhead (Linux eBPF agent)	~15 MB RSS
Memory overhead (Windows ETW agent)	~25 MB RSS

The agent's memory footprint is negligible on modern developer machines. CPU overhead during active AI coding sessions averages 0.3% of a single core, measured on an Intel i7-13700K.

11. Related Work

EDR and endpoint security. CrowdStrike Falcon [4], SentinelOne [1], and Microsoft Defender for Endpoint provide behavioral detection on endpoints. SentinelOne's OneClaw specifically targets AI agent discovery and observability, producing structured telemetry for SIEM ingestion. These tools detect point events but do not provide session-level AI agent attribution or AI-gated detection rules. Our work is complementary — EDR detects known threat patterns; our system provides AI-agent-specific context and behavioral learning.

Prompt injection and LLM security. Greshake et al. [3] demonstrated indirect prompt injection attacks against LLM-integrated applications. Lakera, Rebuff, and Prompt Security provide defense at the API and prompt layer — intercepting and filtering inputs before they reach the model. These tools protect the AI model; our system monitors what the AI agent does after receiving instructions. The threat models are complementary.

UEBA for AI agents. Exabeam [2] extended their behavioral analytics to model AI agents as distinct entities with their own baselines. Current implementations focus on cloud orchestration agents within enterprise platforms

(e.g., Google's agent ecosystem) rather than developer-machine AI coding tools. Our system addresses the developer-laptop gap.

eBPF for security. Cilium Tetragon [5] uses eBPF for runtime security enforcement in Kubernetes environments. KRSI (Kernel Runtime Security Instrumentation) [6] uses BPF LSM programs for mandatory access control. BPFContain [7] uses eBPF for container confinement. Our system uses eBPF specifically for AI agent monitoring, with session-aware attribution that these general-purpose tools do not provide.

AI agent security research. Emerging work has demonstrated that LLM agents can autonomously exploit vulnerabilities [8] and that tool-using agents create novel attack surfaces. Our system provides the runtime monitoring layer that enables organizations to detect when these theoretical attacks manifest in practice.

Agent Detection and Response (ADR). Gen Digital open-sourced Sage [10], a lightweight ADR layer that intercepts AI agent tool calls (bash commands, file writes, URL fetches) via hook systems in Claude Code and Cursor. Sage uses YAML-based heuristic rules and cloud-based URL reputation checks. Microsoft's Agent Governance Toolkit [9] addresses all 10 OWASP agentic AI risks [11] with deterministic policy enforcement. Both tools operate at the application layer, intercepting agent commands before execution. Our system monitors at the kernel level, capturing what actually executes — including actions that bypass application-layer hooks.

AI agent vulnerabilities. Donenfeld et al. [12] demonstrated remote code execution through Claude Code project configuration files (CVE-2025-59536), proving that simply cloning an untrusted repository can compromise a developer's machine. Kaspersky [21] documented a malicious Cursor extension that executed PowerShell scripts to drain cryptocurrency wallets. These real-world attacks validate our threat model and demonstrate the need for kernel-level monitoring independent of application-layer trust.

11.1 Comparison Summary

Table 10: Qualitative comparison with related systems

Capability	Our System	SentinelOne OneClaw [1]	Sage ADR [10]	Microsoft AGT [9]	Exabeam AI Analytics [2]
Monitoring layer	Kernel (eBPF/ETW/ESF)	Kernel (EDR)	Application (hooks)	Application (policy)	Log-based (UEBA)
AI session attribution	Yes (UUID inheritance)	Partial (telemetry)	No	No	Yes (entity modeling)
AI-gated detection rules	13 rules, AI-only	No (general-purpose)	~200 heuristic rules	Policy-based	Behavioral baselines
Multi-step chain detection	11 patterns	No	No	No	Sequence detection
Never-baseline safety list	Yes	No	No	No	No
Cross-platform	Linux, Windows, macOS	Linux, Windows, macOS	Claude Code, Cursor	Agent framework	Cloud platforms
Developer machine focus	Yes	Partial	Yes	No (enterprise agents)	No (cloud agents)
On-premise data	Yes	Yes	Yes (local)	Configurable	Cloud
LLM-powered investigation	Yes (BYOK, 18 tools)	No	No	No	No

12. Discussion and Limitations

Kernel access requirement. Our system requires elevated privileges: root or CAP_BPF on Linux, Administrator on Windows, Full Disk Access on macOS. This is inherent to kernel-level monitoring and cannot be relaxed without losing visibility.

eBPF verifier constraints. The Linux eBPF verifier imposes restrictions on program complexity, stack size (512 bytes), and loop bounds. These constraints occasionally require restructuring monitoring programs in non-obvious ways.

Neo4j dependency. While the system operates without Neo4j, graph features (process tree visualization, attack chain queries, AI label propagation) provide significant investigative value. We recommend Neo4j for production deployments.

Monitoring consent. Deploying runtime monitoring on developer machines raises privacy and trust considerations. Organizations must obtain informed consent and establish clear policies about what is monitored and how data is used. Our on-premise architecture — where all data stays on the organization's infrastructure — mitigates some concerns but does not eliminate the need for transparent communication.

Evasion. A sophisticated attacker who knows the monitoring system is deployed could attempt to evade detection by operating outside the AI agent process tree (e.g., scheduling actions via cron that execute later). Our detection rules for persistence mechanisms (T1053, T1546) partially address this, but targeted evasion of session-attributed monitoring remains an open problem.

Future Work

We plan several extensions to the system:

Response Cycle. A transition from passive monitoring to active enforcement. Once the behavioral baseline system has learned an environment's normal patterns, the agent can be configured to actively block unknown network connections, unauthorized file access, and unapproved port usage. This moves the agent from security camera to security guard.

Team Baselines. Shared behavioral baselines across teams working on the same project. A single security operations center can manage and distribute baselines to all team members, eliminating per-developer noise and enabling organization-wide security policy. Team-level alert configuration allows security teams to define policies once and apply them across the organization.

Container Security. Native Docker and Kubernetes monitoring with pod-level attribution, extending our AI agent tracking into containerized development environments.

Machine Learning Anomaly Detection. Statistical modeling beyond rule-based patterns, enabling detection of novel AI agent behaviors that do not match predefined rules.

Threat Intelligence Integration. Correlation with external threat feeds (MISP, STIX/TAXII) to match local AI agent behavior against known indicators of compromise and threat actor techniques.

Distributed Tracing. OpenTelemetry integration for tracking AI agent activity across service boundaries in distributed architectures.

13. Conclusion

AI coding agents represent a new class of privileged software that operates with full developer permissions yet falls outside the detection envelope of conventional security tools. We presented a kernel-level monitoring system that addresses this gap through platform-native instrumentation, AI agent session attribution, behavioral detection rules, and adaptive baseline learning. The system deploys across Linux, Windows, and macOS, providing unified detection and investigation capabilities regardless of platform.

Our approach demonstrates that effective AI agent security monitoring requires operating at the kernel level — where the truth of system behavior is recorded — rather than at the application or API layer where agents report their own actions. The session UUID inheritance mechanism and AI-gated detection rules eliminate the false positive problem that makes general-purpose endpoint monitoring impractical for this use case.

As AI agents gain more autonomy and deeper system access, the runtime security gap will widen. We believe kernel-level monitoring with behavioral learning provides a foundation for keeping that gap manageable — enabling organizations to adopt AI tools aggressively while maintaining the visibility that security governance requires.

References

- [1] SentinelOne. "OneClaw: AI Agent Discovery and Security Observability." SentinelOne Technical Blog, 2025.
- [2] Exabeam. "AI Agent Behavior Analytics: Modeling Autonomous Agents as Security Entities." Exabeam Research, 2025.
- [3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection." In AISec Workshop at ACM CCS, 2023.
- [4] CrowdStrike. "Falcon Platform: Endpoint Detection and Response Architecture." CrowdStrike Technical Documentation, 2024.
- [5] Isovalent/Cilium. "Tetragon: eBPF-Based Security Observability and Runtime Enforcement." <https://tetragon.io>, 2024.
- [6] KP Singh, F. Revest, B. Jackman. "KRSI: Kernel Runtime Security Instrumentation." Linux Kernel Documentation, kernel.org, 2020.
- [7] W. Findlay, A. Somayaji, and D. Barrera. "BPFContain: Fixing the Soft Underbelly of Container Security with BPF." arXiv preprint, 2022.
- [8] R. Fang, R. Bindu, A. Gupta, and D. Kang. "LLM Agents Can Autonomously Hack Websites." arXiv preprint arXiv:2402.06664, 2024.
- [9] Microsoft. "Introducing the Agent Governance Toolkit: Open-source Runtime Security for AI Agents." Microsoft Open Source Blog, April 2026.
- [10] Gen Digital. "Sage: Agent Detection & Response (ADR) Layer for AI Agents." <https://github.com/gendigitalinc/sage>, 2026.
- [11] OWASP. "OWASP Top 10 for Agentic Applications 2026." OWASP Foundation, December 2025.
- [12] A. Donenfeld et al. "RCE and API Token Exfiltration through Claude Code Project Files (CVE-2025-59536, CVE-2026-21852)." Check Point Research, 2026.

- [13] A. Karpathy. "Vibe Coding." Twitter/X post, February 2025. The term describes fully AI-delegated coding where developers "forget that the code even exists."
- [14] GitHub. "GitHub Expands Application Security Coverage with AI-Powered Detections." GitHub Blog, 2026.
- [15] GitHub. "Secret Scanning for AI Coding Agents via GitHub MCP Server." GitHub Blog, March 2026.
- [16] OpenAI. "Introducing Aardvark: OpenAI's Agentic Security Researcher." OpenAI Blog, 2026.
- [17] H.-M. Darley. "Runtime: The New Frontier of AI Agent Security." CSO Online, 2026.
- [18] D. Kang, X. Li, I. Stoica, C. Guestrin, M. Zaharia, and T. Hashimoto. "Exploiting Novel GPT-4 APIs." arXiv preprint arXiv:2312.14302, 2023.
- [19] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson. "Universal and Transferable Adversarial Attacks on Aligned Language Models." arXiv preprint arXiv:2307.15043, 2023.
- [20] S. Peisert et al. "Perspectives on the SolarWinds Incident." IEEE Security & Privacy, 2021.
- [21] Kaspersky. "Open-Source Package for Cursor AI Turned Into a Crypto Heist." Securelist, 2026.